TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN – ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH

CE119 – Thực hành Kiến trúc Máy tính

LAB 02

MẢNG, CHUỖI, THỦ TỤC VÀ NGĂN XẾP

Thời lượng: 180 phút · Công cụ: MARS 4.5

Họ và tên	Dương Gia Thịnh	**MSSV**	25521759
Lớp	VMTN2025	**Ngày thực hành**	20/10/2026
Tự chấm (/10)	10/10	**Liên kết GitHub**	https://github.com/TDuong512

1. TỔNG QUAN BUỔI THỰC HÀNH

Mục đích

Buổi thực hành xây dựng một bộ tiện ích nhỏ cho thiết bị nhúng: thống kê mảng điểm số và kiểm tra cặp ngoặc trong chuỗi bằng ngăn xếp (stack). Sinh viên làm việc trực tiếp với bộ nhớ (địa chỉ, offset, căn lề, thứ tự byte); được hướng dẫn cách viết thủ tục, tức là đơn vị mã có tên, nhận tham số và trả kết quả theo quy ước thanh ghi; và dùng ngăn xếp như một cấu trúc dữ liệu "vào sau, ra trước", quan sát hoạt động của nó trên MARS thay vì chỉ học thuộc quy tắc.

Sau buổi thực hành, sinh viên có thể

- tính đúng địa chỉ và offset khi truy cập mảng word, mảng byte và chuỗi bằng `lw`, `sw`, `lb`, `lbu`, `sb`; giải thích kết quả quan sát được trong Data Segment (căn lề, thứ tự byte, mở rộng dấu);
- giải thích cơ chế gọi và trả về của thủ tục (`jal`, `jr $ra`, thanh ghi `$ra`); viết thủ tục lá nhận mảng làm tham số và trả về một hoặc hai giá trị theo quy ước `$a`, `$v`, `$t`;
- thực hiện thao tác push và pop bằng `addi`, `sw`, `lw` trên `$sp`; giải thích vì sao ngăn xếp phù hợp với bài toán kiểm tra ngoặc;
- giữ `$sp` cân bằng trên mọi đường thoát của một thủ tục;
- trực quan hóa dữ liệu thống kê bằng ký tự văn bản (phần mở rộng).

Bảng 1. *Phân bổ nội dung, thời gian và điểm*

Phần	Nội dung	Thời gian gợi ý	Điểm
Phần 0	Chuẩn bị môi trường, tham số cá nhân	10 phút	–
TN1	Bản đồ bộ nhớ: địa chỉ, căn lề, thứ tự byte, `lb`/`lbu`	20 phút	1,5
TN2	Thủ tục và mảng: hướng dẫn thủ tục, thống kê điểm bằng thủ tục lá	55 phút	3,0
TN3	Ngăn xếp: kiểm tra ngoặc cân bằng	60 phút	3,5
Kết thúc	Nộp bài, phản tư, vấn đáp	20 phút	2,0
Mở rộng	Trực quan hóa phổ điểm bằng ký tự `|` (dành cho sinh viên hoàn thành sớm)	15 phút	+1,0
Tổng		**180 phút**	**10,0**

Quy ước làm bài

- **Ô nền vàng nhạt** là chỗ điền câu trả lời (chữ nhập vào ô có màu đỏ); **khung nét đứt** là nơi dán ảnh chụp màn hình MARS (ảnh phải thấy rõ thanh ghi hoặc vùng nhớ liên quan).
- Mỗi thí nghiệm theo chu trình **DỰ ĐOÁN → QUAN SÁT → GIẢI THÍCH**. Điền dự đoán **trước** khi chạy. Dự đoán sai không bị trừ điểm; phần giải thích mới được chấm.
- Câu hỏi được gắn mức nhận thức: Hiểu, Áp dụng, Phân tích, Đánh giá.
- Chương trình nộp là **một file .asm duy nhất**. Mỗi thủ tục có chú thích đầu: tham số, giá trị trả về, thanh ghi sử dụng. Chuỗi in ra và chú thích dùng ASCII không dấu.
- Mọi giá trị địa chỉ và thanh ghi ghi ở dạng thập lục phân (hex) 8 chữ số, trừ khi đề yêu cầu khác.

PHẦN 0 – CHUẨN BỊ MÔI TRƯỜNG (10 phút)

Bảng 2. *Kỹ năng cần thực hiện được trước khi làm thí nghiệm*

☐	Kỹ năng	Gợi ý
☐	Bật khởi tạo PC tại nhãn `main`	Menu Settings → "Initialize Program Counter to global 'main' if defined"
☐	Hiển thị Data Segment dạng hex và ASCII	Các ô chọn ở cuối cửa sổ Data Segment: Hexadecimal Values, ASCII
☐	Chuyển vùng xem bộ nhớ sang `.data` và sang "current $sp"	Danh sách thả xuống ở cuối cửa sổ Data Segment
☐	Xem địa chỉ các nhãn	Ô Labels ở cửa sổ Execute
☐	Chạy từng bước, lùi bước, đặt breakpoint	Các nút Step, Backstep; ô Bkpt ở cột đầu của Text Segment

Bước 0.1 – Kiểm tra nhanh

Nhập và chạy chương trình sau để làm quen với syscall in số thập lục phân (mã 34) và in ký tự (mã 11). Ghi lại hai giá trị in ra; giá trị `$sp` được dùng lại ở TN3.

```
.data
w1:   .word 0
.text
main:
    move $a0, $sp
    li   $v0, 34
    syscall
    li   $a0, 10
    li   $v0, 11
    syscall
    la   $a0, w1
    li   $v0, 34
    syscall
    li   $v0, 10
    syscall
```

Bảng 3. *Giá trị đo ở Bước 0.1*

Đại lượng	Giá trị (hex)
`$sp` khi chương trình bắt đầu	0x7fffeffc
Địa chỉ nhãn `w1`	0x10010000

Bước 0.2 – Tham số cá nhân

Gọi **d** là chữ số cuối cùng của MSSV. Điền các tham số cá nhân dùng trong các thí nghiệm. Cơ sở của mảng điểm là dãy **A0 = {6, 3, 8, 9, 2, 5, 7, 2, 6, 9, 4, 3}** (12 phần tử); mảng điểm của bạn là S[i] = (A0[i] + d) mod 11, mọi phần tử đều thuộc [0, 10].

Bảng 4. *Tham số cá nhân*

Tham số	Công thức	Giá trị của bạn
d	chữ số cuối của MSSV	9
Mảng điểm S[0..11]	S[i] = (A0[i] + d) mod 11	{4, 1, 6, 7, 0, 3, 5, 0, 4, 7, 2, 1}
k	d + 3	12

Quy ước đặt tên file nộp: `CE119_Lab02_<MSSV>.asm` cho mã nguồn và `CE119_Lab02_<MSSV>.docx` cho báo cáo này (đã điền).

TN1 – BẢN ĐỒ BỘ NHỚ (20 phút · 1,5 điểm)

Tình huống

Một bản ghi cấu hình của thiết bị nhúng gồm một word, ba byte điều khiển và một chuỗi định danh. Trước khi viết thủ tục duyệt mảng và chuỗi, cần xác định chính xác vị trí, giá trị và thứ tự của từng byte trong bộ nhớ.

Nhắc lại hai lệnh nạp byte

- `lb` nạp một byte rồi **mở rộng dấu** (sign-extend): bit 7 của byte được chép vào 24 bit cao.
- `lbu` nạp một byte rồi **mở rộng không dấu** (zero-extend): 24 bit cao bằng 0.

Bước 1.1. Nhập chương trình sau, assemble, và mở Data Segment (bật hex và ASCII).

```
.data
w1:  .word 0x8A7B6C5D
b1:  .byte 0x80, 0x7F, 0xFF
     .align 2
w2:  .word 1
s1:  .asciiz "IT012"
.text
main:
    la   $s0, w1
    lw   $t0, 0($s0)      # (a)
    lb   $t0, 0($s0)      # (b)
    lb   $t0, 3($s0)      # (c)
    lbu  $t0, 3($s0)      # (d)
    lb   $t0, 4($s0)      # (e)
    lbu  $t0, 4($s0)      # (f)
    lw   $t0, 4($s0)      # (g)
    li   $t1, 0xAB
    sb   $t1, 1($s0)      # (h)
    lw   $t0, 0($s0)      # (i)
    li   $v0, 10
    syscall
```

Bước 1.2 (DỰ ĐOÁN → QUAN SÁT). Điền Bảng 5 với địa chỉ dự đoán của từng nhãn (vùng `.data` bắt đầu tại `0x10010000`), sau đó đối chiếu với cửa sổ Labels.

Bảng 5. *Địa chỉ các nhãn*

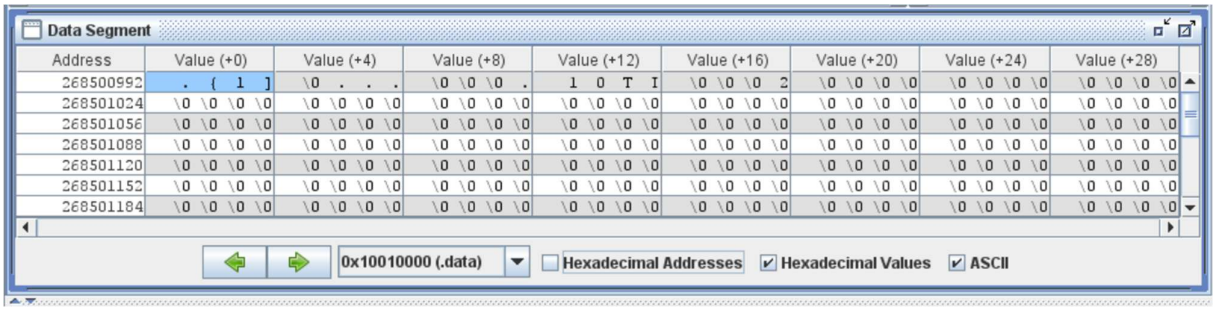
Nhãn	Dự đoán	Quan sát
`w1`	0x10010000	0x10010000
`b1`	0x10010004	0x10010004
`w2`	0x10010008	0x10010008
`s1`	0x1001000c	0x1001000c

Bước 1.3 (DỰ ĐOÁN → QUAN SÁT). Với mỗi dòng (a)–(i), ghi giá trị `$t0` mà bạn dự đoán theo mô hình bộ nhớ được trình bày trong slide Chương 2, rồi chạy từng bước (Step) và ghi giá trị quan sát. Dòng (i) đọc lại word tại `w1` sau khi (h) đã ghi.

Bảng 6. *Giá trị* `$t0` *sau mỗi lệnh*

Dòng	Lệnh	Dự đoán `$t0`	Quan sát `$t0`	Khớp (✓/✗)
(a)	`lw $t0, 0($s0)`	0x8a7b6c5d	0x8a7b6c5d	Khớp
(b)	`lb $t0, 0($s0)`	0x0000005d	0x0000005d	Khớp
(c)	`lb $t0, 3($s0)`	0xffffff8a	0xffffff8a	Khớp
(d)	`lbu $t0, 3($s0)`	0x0000008a	0x0000008a	Khớp
(e)	`lb $t0, 4($s0)`	0xffffff80	0xffffff80	Khớp
(f)	`lbu $t0, 4($s0)`	0x00000080	0x00000080	Khớp
(g)	`lw $t0, 4($s0)`	0x00ff7f80	0x00ff7f80	Khớp
(h)+(i)	`sb $t1, 1($s0)` rồi `lw $t0, 0($s0)`	0x8a7bab5d	0x8a7bab5d	Khớp

Dán ảnh Data Segment (hex và ASCII) vùng từ `w1` đến hết `s1`, trước khi chạy dòng (h):



"268500992 = 0x10010000"

Câu 1.1 (Phân tích). Giải thích vì sao địa chỉ của `w2` không nằm ngay sau byte cuối của `b1`. Byte nằm ở khoảng trống đó có giá trị bao nhiêu và do đâu mà có?

Vì b1 có 3 byte chiếm vị trí từ 0x10010004 – 0x10010006 và có .align = 2 tức là sau khi khởi tạo giá trị cho b1 thì con trỏ phải trỏ tới ô tiếp theo chia hết cho 4 vì 7 không chia hết cho 4 nên assmebler chèn 1 byte đệm có giá trị 0x00 do vùng .data được khỏi tạo bằng 0. Sau đó w2 bắt đầu ở 0x10010008

Câu 1.2 (Phân tích). So sánh kết quả của (e) và (f). Dựa trên phần số có dấu và không dấu trong slide, giải thích cơ chế tạo ra từng kết quả.

Kết quả đều đọc một byte 0x80 nhưng lệnh lb sẽ cần xét xem bit 7 có phải là số âm hay không , nếu là âm tức bằng 1 thì sẽ chép 1 vào 24 bit cao cho ra 0xffffff80 . Lệnh lbu thì là số không dấu do đó 24 bit cao chứa số 0 và kết quả cho ra là 0x00000080

Câu 1.3 (Phân tích). Đối chiếu cột dự đoán với cột quan sát ở (b), (c), (g) và (i). Nếu có sai khác, hãy chỉ ra chính xác sai khác nằm ở đâu và giải thích nguyên nhân bằng dữ liệu bạn thấy trong Data Segment.

Dự đoán và quan sát khớp ở cả (b), (c), (g), (i) vì em đã dự đoán theo little-endian. Data Segment cho thấy w1 được lưu thành các byte 5D 6C 7B 8A từ địa chỉ thấp đến cao, nên offset 0 là 0x5D (b), offset 3 là 0x8A (c), 4 byte từ 0x10010004 là 80 7F FF 00, ghép lại được 0x00FF7F80 (g), và sb vào offset 1 thay byte 0x6C thành 0xAB (i).

Bước 1.4 (DỰ ĐOÁN → QUAN SÁT). Thêm dòng `lw $t0, 1($s0)` ngay trước `li $v0, 10`, assemble và chạy. Ghi lại hiện tượng.

Câu 1.4 (Áp dụng). Chép nguyên văn thông báo của MARS và nêu điều kiện của địa chỉ bị vi phạm. Với một mảng word có địa chỉ nền `$s3`, công thức địa chỉ phần tử `i` đảm bảo điều kiện này là gì?

Error in C:\Users\THINH PC\Documents\Kien truc may tinh - TH\Computer-Architecture\Application\tn1_example.jar line 20: Runtime exception at 0x00400030: fetch address not aligned on word boundary 0x10010001

Điều kiện vi phạm là : lệnh lw yêu cầu địa chỉ phải chia hết cho 4 nhưng 1($s0) tạo ra địa chỉ 0x10010001 không thỏa mãn. Công thức giúp đảm bảo không vi phạm là địa chỉ phần tử thứ i của mảng có địa chỉ nền $s3 phải là : $s3 + (i*4)

TN2 – THỦ TỤC VÀ MẢNG: THỐNG KÊ ĐIỂM (55 phút · 3,0 điểm)

Tình huống

Hệ thống quản lý lớp học cần thống kê mảng điểm nguyên (thang 0–10) trước khi xuất báo cáo: điểm thấp nhất, điểm cao nhất và số sinh viên trong từng nhóm xếp loại. Phần mềm được chia thành các thủ tục nhỏ, mỗi thủ tục làm một việc và có thể kiểm thử riêng. Trong thí nghiệm này, trước hết bạn làm quen với thủ tục qua hai ví dụ mẫu chạy được, sau đó tự viết hai thủ tục thống kê.

A. Kiến thức nền: thủ tục

Thủ tục (procedure) là một đoạn mã có tên (nhãn), được gọi từ nhiều nơi trong chương trình và khi chạy xong thì quay về đúng chỗ đã gọi. Dùng thủ tục giúp chia chương trình lớn thành các phần nhỏ, kiểm thử riêng từng phần và tái sử dụng.

Việc gọi và quay về do hai lệnh đảm nhiệm:

- `jal nhãn` (jump and link): ghi địa chỉ của lệnh nằm **ngay sau** `jal` vào thanh ghi `$ra` (return address), rồi nhảy đến `nhãn`.
- `jr $ra` (jump register): nhảy đến địa chỉ đang chứa trong `$ra`, tức là quay về lệnh nằm ngay sau `jal`.

Người gọi và thủ tục trao đổi dữ liệu qua thanh ghi theo một quy ước chung:

Quy ước thanh ghi khi gọi thủ tục

- `$a0`–`$a3`: tham số truyền vào (tối đa bốn). Thủ tục được phép thay đổi các thanh ghi này.
- `$v0`, `$v1`: giá trị trả về. (`$v0` cũng chứa mã syscall, `$a0` cũng chứa tham số của syscall: lệnh `syscall` làm thay đổi chúng.)
- `$t0`–`$t9`: thanh ghi tạm. Thủ tục dùng tự do; người gọi **không được** trông cậy vào giá trị của chúng sau lời gọi.
- `$s0`–`$s7`: thanh ghi "giữ nguyên qua lời gọi". Thủ tục nào dùng đến chúng phải trả lại nguyên giá trị khi return. Các thủ tục trong bài này không dùng `$s`, nên chương trình chính có thể cất giá trị vào `$s0`–`$s7` để dùng sau các lời gọi.
- `$ra`: địa chỉ trả về, do `jal` ghi.

Thủ tục chỉ dùng `$a`, `$v`, `$t` và **không gọi thủ tục khác** (không có `jal` bên trong) gọi là **thủ tục lá (leaf procedure)**. Với thủ tục lá, `$ra` không bị thay đổi nên chỉ cần `jr $ra` ở cuối. Trường hợp thủ tục gọi thủ tục khác cần thêm kỹ thuật lưu `$ra`, được học ở bài thực hành sau; trong bài này mọi thủ tục đều là thủ tục lá.

Quy tắc viết: mỗi thủ tục bắt đầu bằng một nhãn riêng và kết thúc bằng `jr $ra`; nhãn bên trong thủ tục (vòng lặp, nhánh) nên có tiền tố là tên thủ tục (ví dụ `max2_end`) vì nhãn phải duy nhất trong cả file; chương trình chính phải kết thúc bằng syscall thoát (mã 10) **trước** đoạn mã thủ tục, nếu không chương trình sẽ "chạy lọt" vào thủ tục.

B. Ví dụ mẫu 1: thủ tục có hai tham số và một kết quả

Thủ tục `max2` nhận hai số nguyên có dấu trong `$a0`, `$a1` và trả về số lớn hơn trong `$v0`. Chương trình chính nạp tham số, gọi thủ tục bằng `jal`, rồi in kết quả.

```
.text
main:
    li   $a0, 17
    li   $a1, 25
    jal  max2
    move $a0, $v0
    li   $v0, 1
    syscall
    li   $v0, 10
    syscall

# max2: tra ve so lon hon trong hai so nguyen co dau
# Tham so : $a0 = a, $a1 = b
# Tra ve  : $v0 = max(a, b)
max2:
    move $v0, $a0
    slt  $t0, $a0, $a1
    beq  $t0, $zero, max2_end
    move $v0, $a1
max2_end:
    jr   $ra
```

Bước 2.1 (DỰ ĐOÁN → QUAN SÁT). Nhập chương trình, assemble và xem cột Address trong Text Segment. Dựa vào địa chỉ các lệnh, điền cột "Dự đoán" của Bảng 7; sau đó chạy từng bước (Step) và điền cột "Quan sát".

Bảng 7. *Gọi và trả về của* `max2`

Thời điểm	Dự đoán	Quan sát
PC ngay sau khi `jal max2` thực thi	0x00400020	0x00400020
`$ra` ngay sau khi `jal max2` thực thi	0x0040000c	0x0040000c
`$v0` ngay trước khi `jr $ra` thực thi	0x00000019	0x00000019
PC ngay sau khi `jr $ra` thực thi	0x0040000c	0x0040000c

Câu 2.1 (Hiểu). Dựa vào Bảng 7, mô tả bằng lời cách `jal` và `jr $ra` phối hợp để chương trình quay về đúng chỗ. Nếu chương trình chính gọi `max2` ở hai vị trí khác nhau thì giá trị `$ra` ở hai lần gọi có giống nhau không? Điều đó giúp một thủ tục được gọi từ nhiều nơi như thế nào?

Lệnh jal làm 2 việc là nhảy tới nhãn mình muốn và lưu địa chỉ của lệnh kế tiếp của jal vào thanh ghi $ra. Lệnh jr $ra dùng lại địa chỉ đã lưu vào thanh ghi $ra để thực hiện lệnh nhảy lại tới vị trí đó. Ở mỗi lần jal chạy, nó ghi đè $ra bằng địa chỉ sau chính lệnh jal đó, nên $ra luôn chứa địa chỉ quay về của lần gọi gần nhất. Nhờ vậy, thân thủ tục không cần biết ai gọi mình: cùng một lệnh jr $ra luôn quay về đúng nơi gọi. Do đó một thủ tục có thể được gọi từ nhiều nơi mà chỉ cần viết một lần.

C. Ví dụ mẫu 2: truyền mảng cho thủ tục

Một thủ tục nhận mảng thông qua **địa chỉ đầu mảng** và **số phần tử**. Thủ tục `sum_array` dưới đây duyệt mảng bằng con trỏ: sau mỗi phần tử kiểu word, con trỏ `$a0` tăng thêm 4. (Cách khác là giữ chỉ số `i` và tính địa chỉ bằng `sll` và `add` như trong slide.)

```
.data
arr: .word 4, 9, 1, 6
.text
main:
    la   $a0, arr
    li   $a1, 4
    jal  sum_array
    move $a0, $v0
    li   $v0, 1
    syscall
    li   $v0, 10
    syscall

# sum_array: tong cac phan tu cua mang word
# Tham so : $a0 = dia chi mang, $a1 = n (n >= 0)
# Tra ve  : $v0 = tong
sum_array:
    move $v0, $zero
sa_loop:
    beq  $a1, $zero, sa_done
    lw   $t0, 0($a0)
    add  $v0, $v0, $t0
    addi $a0, $a0, 4
    addi $a1, $a1, -1
    j    sa_loop
sa_done:
    jr   $ra
```

Bước 2.2 (DỰ ĐOÁN → QUAN SÁT). Dự đoán các đại lượng trong Bảng 8. Chạy chương trình, đặt breakpoint tại lệnh `move $a0, $v0` (ngay sau lời gọi) để đọc `$a0` và `$a1` khi thủ tục vừa trả về, rồi chạy tiếp để xem số in ra.

Bảng 8. *Kết quả của* `sum_array`

Đại lượng	Dự đoán	Quan sát
Số in ra	20	20
`$a0` khi thủ tục vừa trả về (hex)	0x10010010	0x10010010
`$a1` khi thủ tục vừa trả về (hex)	0x00000000	0x00000000

Câu 2.2 (Phân tích). So sánh `$a0` và `$a1` khi thủ tục vừa trả về với giá trị lúc gọi. Quy ước thanh ghi cho phép điều này hay không? Hệ quả đối với chương trình chính là gì nếu nó cần dùng lại địa chỉ mảng sau lời gọi, và có những cách nào để xử lý?

Lúc gọi $a0 = 0x10010000 và $a1 = 0x00000004; khi trả về $a0 = 0x10010010 (đã trỏ quá phần tử cuối) và $a1 = 0x00000000, vì thủ tục dùng chính hai thanh ghi này làm con trỏ và bộ đếm. Quy ước cho phép điều này, vì $a0–$a3 là thanh ghi thủ tục được tự do thay đổi. Hệ quả: nếu main dùng lại $a0, $a1 cho lời gọi tiếp theo trên cùng mảng thì thủ tục sau sẽ thấy n = 0 và thoát ngay, hoặc (nếu chỉ nạp lại n) sẽ đọc từ 0x10010010, nằm ngoài mảng, cho kết quả sai. Cách xử lý: (1) người gọi nạp lại tham số bằng la/li trước mỗi lời gọi; (2) người gọi cất địa chỉ mảng và n vào $s0, $s1 (giữ nguyên qua lời gọi) rồi move sang $a0, $a1 mỗi lần gọi; (3) thủ tục chép $a0, $a1 sang $t0, $t1 và chỉ thay đổi các thanh ghi $t.

D. Nhiệm vụ: thống kê điểm

Nhóm xếp loại: nhóm 0 gồm điểm 0–3; nhóm 1 gồm điểm 4–6; nhóm 2 gồm điểm 7–8; nhóm 3 gồm điểm 9–10. Bảng tra `grade_tab` có 11 phần tử kiểu byte, phần tử thứ `s` cho biết nhóm của điểm `s`; mảng đếm `hist` có 4 word, khởi tạo bằng 0.

Bước 2.3. Khai báo dữ liệu (mảng điểm cá nhân, `grade_tab`, `hist`) và cài đặt hai thủ tục theo đặc tả giao diện sau. Không có mã khung; tự thiết kế thuật toán.

Bảng 9. *Đặc tả giao diện thủ tục*

Thủ tục	Tham số	Kết quả và ràng buộc
`stats`	`$a0` = địa chỉ mảng word `$a1` = n (n ≥ 1)	`$v0` = giá trị nhỏ nhất, `$v1` = giá trị lớn nhất. Một lượt duyệt mảng; không ghi bộ nhớ; là thủ tục lá.

`histogram`	`$a0` = địa chỉ mảng điểm
	`$a1` = n
	`$a2` = địa chỉ `grade_tab`
	`$a3` = địa chỉ `hist`
		Với mỗi điểm `s`, tăng `hist[grade_tab[s]]` thêm 1. Không trả giá trị. Bắt buộc tra bảng bằng `lbu`, không so sánh điểm với ngưỡng. Là thủ tục lá.

Bước 2.4. Viết chương trình chính: nạp tham số, gọi hai thủ tục, in kết quả đúng định dạng sau (hai dòng):

```
min=<giá trị> max=<giá trị>
g0=<số> g1=<số> g2=<số> g3=<số>
```

Bước 2.5 (DỰ ĐOÁN → QUAN SÁT). Với mỗi bộ dữ liệu trong Bảng 10, tính tay kết quả trước, sau đó chạy chương trình. Bộ T2 gồm một phần tử {10}; bộ T3 gồm năm phần tử cùng bằng 4.

Bảng 10. *Kết quả thống kê*

Bộ dữ liệu	min / max (tính tay)	g0 g1 g2 g3 (tính tay)	Chương trình in ra
T1: mảng cá nhân (n = 12)	0/7	6 4 2 0	min=0 max=7 g0=6 g1=4 g2=2 g3=0
T2: {10} (n = 1)	10/10	0 0 0 1	min=10 max=10 g0=0 g1=0 g2=0 g3=1
T3: {4, 4, 4, 4, 4} (n = 5)	4/4	0 5 0 0	min=4 max=4 g0=0 g1=5 g2=0 g3=0

Câu 2.3 (Phân tích). Sau khi `histogram` trả về, người gọi có thể trông cậy vào giá trị của `$a0`–`$a3` hay không? Giải thích bằng quy ước thanh ghi và chỉ ra thanh ghi nào trong bài của bạn đã bị thay đổi. Vì sao hai thủ tục trên không cần lưu `$ra`?

Không nên trong cậy do các thanh ghi $a0-$a3 theo quy ước được phép thay đổi các tham số. Trong bài của em histogram chép $a0,$a1 vào $t0,$t1 trước khi duyệt nên thực tế các thanh ghi $a0-$a3 không bị thay đổi. Các thanh ghi thay đổi là $t0-$t7.Hai thủ tục trên không cần lưu $ra vì chúng là thủ tục lá tức là không có lệnh jal nào bên trong nên $ra do main ghi vẫn được giữ nguyên đến lúc jr $ra

Câu 2.4 (Áp dụng). Đếm số lệnh thực thi trong thân vòng lặp của `histogram` cho một phần tử (kể cả lệnh nhảy). Chỉ ra lệnh nào chuyển chỉ số phần tử thành offset byte đối với mảng word, và vì sao truy cập `grade_tab` không cần lệnh tương tự.

Thân vòng lặp histogram thực thi 12 lệnh cho mỗi phần tử, tính cả beq kiểm tra dừng và j quay lại. Lệnh sll $t5,$t4,2 chuyển chỉ số nhóm g thành offset byte 4g của mảng word hist, vì mỗi word chiếm 4 byte nên hist[g] nằm tại hist + 4g. Truy cập grade_tab không cần lệnh tương tự vì đây là mảng byte: mỗi phần tử chiếm 1 byte nên offset bằng đúng chỉ số s, chỉ cần add $t3,$a2,$t2.

Câu 2.5 (Đánh giá). Giả sử dữ liệu nhập có một điểm không hợp lệ bằng 11. Phân tích điều gì xảy ra trong `histogram` của bạn (byte được đọc từ đâu, ghi vào đâu). Đề xuất một cách bảo vệ và ước lượng chi phí của nó theo số lệnh thêm vào trên mỗi phần tử.

Với s = 11, lệnh add $t3,$a2,$t2 cho địa chỉ grade_tab + 11 = 0x1001003B, nằm ngoài grade_tab (chỉ số hợp lệ 0..10), đúng vào byte đệm do .align 2 chèn trước hist. Byte này bằng 0x00 nên lbu trả về g = 0, và chương trình tăng hist[0] tại 0x1001003C. Điểm 11 bị đếm nhầm vào nhóm 0 mà không có lỗi nào được báo. Thử với mảng có phần tử 11, chương trình in g0=7 g1=3. Nếu s ≥ 12, lbu đọc vào chính các byte của hist nên g có thể lớn hơn 3, và sw sẽ ghi ra ngoài hist, đè lên các chuỗi phía sau. Cách bảo vệ: thêm sltiu $t8,$t2,11 và beq $t8,$zero,histogram_next để bỏ qua phần tử ngoài [0, 10] Chi phí: thêm 2 lệnh mỗi phần tử (12 lên 14 lệnh, khoảng +17%).

TN3 – NGĂN XẾP: KIỂM TRA NGOẶC CÂN BẰNG (60 phút · 3,5 điểm)

Tình huống

Một thiết bị nhúng (ví dụ bộ điều khiển công nghiệp) cho phép người vận hành nhập công thức hiệu chỉnh cảm biến dưới dạng văn bản, chẳng hạn `{k * (a[i] + b)}`. Trước khi tính toán, phần mềm của thiết bị phải kiểm tra các cặp ngoặc `( )`, `[ ]`, `{ }` có hợp lệ hay không. Nếu thiếu ngoặc hoặc lẫn loại ngoặc mà vẫn tính, công thức bị hiểu sai và thiết bị cho kết quả sai mà không báo lỗi. Trình soạn thảo mã nguồn và trình biên dịch cũng phải làm việc tương tự.

Bài toán. Nhận một chuỗi ký tự và trả lời hai câu hỏi: chuỗi có hợp lệ không, và nếu không thì sai ở ký tự thứ mấy, sai theo kiểu nào. Chuỗi hợp lệ khi mỗi ngoặc mở được đóng bởi đúng ngoặc cùng loại và các cặp lồng nhau đúng thứ tự, nghĩa là cặp mở sau phải được đóng trước. Mọi ký tự khác ngoặc đều được bỏ qua.

Vì sao chỉ đếm số ngoặc là không đủ? Chuỗi `[(])` có số `[` bằng số `]` và số `(` bằng số `)`, nhưng vẫn sai: ngoặc `(` mở sau cùng lại chưa đóng thì đã gặp `]`. Muốn phát hiện, phải nhớ **thứ tự** các ngoặc mở chưa đóng và luôn đối chiếu ngoặc đóng với ngoặc mở **gần nhất**. Đó chính là tính chất "vào sau, ra trước" của ngăn xếp.

A. Kiến thức nền: ngăn xếp

Ngăn xếp (stack) là cấu trúc dữ liệu hoạt động theo nguyên tắc **LIFO** (last in, first out): phần tử đưa vào sau cùng là phần tử lấy ra đầu tiên, giống chồng đĩa: chỉ đặt thêm hoặc lấy đi ở đỉnh. Hai thao tác cơ bản là **push** (đẩy một phần tử lên đỉnh) và **pop** (lấy phần tử ở đỉnh ra). Ngăn xếp được dùng rất nhiều trong máy tính: lưu dữ liệu tạm khi thủ tục chạy, tính biểu thức, hoàn tác thao tác trong trình soạn thảo, kiểm tra cú pháp. Trong bài này ngăn xếp đóng vai trò cấu trúc dữ liệu của thuật toán.

Trong MIPS, ngăn xếp là một vùng bộ nhớ thông thường được quản lý bằng thanh ghi `$sp` (stack pointer), luôn trỏ tới phần tử ở **đỉnh**. Một số đặc điểm cần nhớ:

- Ngăn xếp **mở rộng về phía địa chỉ thấp**: push làm `$sp` giảm, pop làm `$sp` tăng.
- Mỗi phần tử trong bài này là một word (4 byte), nên `$sp` luôn là bội số của 4.
- Tập lệnh MIPS không có lệnh push hoặc pop riêng; mỗi thao tác gồm **hai lệnh** như bảng dưới. Thứ tự hai lệnh quan trọng: push phải dịch `$sp` trước rồi mới ghi; pop phải đọc trước rồi mới dịch `$sp` lên.

```
# push $t0                  # pop vao $t0
addi $sp, $sp, -4           lw   $t0, 0($sp)
sw   $t0, 0($sp)            addi $sp, $sp, 4
```

Quy tắc cân bằng: tại mọi lối ra của thủ tục, `$sp` phải bằng đúng giá trị lúc thủ tục được gọi. Nếu không, ngăn xếp của người gọi bị lệch và lỗi thường xuất hiện ở nơi cách xa chỗ gây lỗi.

B. Ví dụ mẫu: push và pop

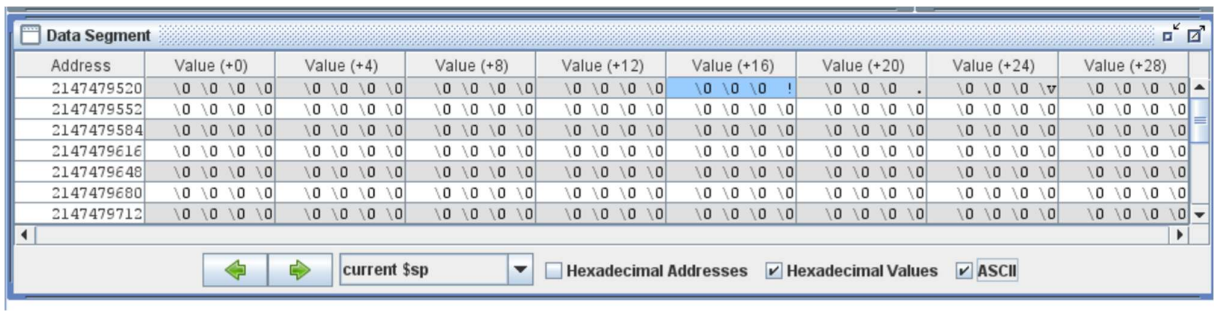
Chương trình dưới đây đẩy ba giá trị 11, 22, 33 lên ngăn xếp rồi lấy ra lần lượt vào `$t3`, `$t4`, `$t5`.

```
.text
main:
    li   $t0, 11
    li   $t1, 22
    li   $t2, 33
    addi $sp, $sp, -4
    sw   $t0, 0($sp)        # push 11
    addi $sp, $sp, -4
    sw   $t1, 0($sp)        # push 22
    addi $sp, $sp, -4
    sw   $t2, 0($sp)        # push 33
    lw   $t3, 0($sp)        # pop
    addi $sp, $sp, 4
    lw   $t4, 0($sp)        # pop
    addi $sp, $sp, 4
    lw   $t5, 0($sp)        # pop
    addi $sp, $sp, 4
    li   $v0, 10
    syscall
```

Bước 3.1 (DỰ ĐOÁN → QUAN SÁT). Gọi S_0 là giá trị `$sp` lúc bắt đầu (đã ghi ở Bảng 3). Dự đoán `$sp` và nội dung word tại `$sp` sau mỗi thao tác trong Bảng 11, rồi chạy từng bước, chuyển Data Segment sang "current $sp" và ghi giá trị quan sát. Dán ảnh Data Segment sau thao tác pop cuối cùng.

Bảng 11. *Trạng thái ngăn xếp sau từng thao tác*

Sau thao tác	`$sp` dự đoán	`$sp` quan sát	Word tại `$sp` (quan sát)
push 11	0x7fffeff8	0x7fffeff8	0x0000000b
push 22	0x7fffeff4	0x7fffeff4	0x00000016
push 33	0x7fffeff0	0x7fffeff0	0x00000021
pop (vào `$t3`)	0x7fffeff4	0x7fffeff4	0x00000016
pop (vào `$t4`)	0x7fffeff8	0x7fffeff8	0x0000000b
pop (vào `$t5`)	0x7fffeffc	0x7fffeffc	0x00000000



"2147479520 = 0x7fffefe0"

Câu 3.1 (Phân tích). Sau khi pop xong, ba giá trị 11, 22, 33 còn nằm trong bộ nhớ hay đã bị xóa? Giải thích ý nghĩa của thao tác "pop" ở mức lệnh, và nêu một hệ quả đối với việc gỡ lỗi hoặc bảo mật dữ liệu.

Sau khi pop xong, ba giá trị 11, 22, 33 vẫn còn nằm trong bộ nhớ, không bị xóa đi. Ở mức lệnh, "pop" thực chất chỉ là lệnh lw sao chép dữ liệu từ bộ nhớ vào thanh ghi, sau đó addi tăng địa chỉ $sp lên, bỏ qua vùng nhớ đó chứ không ghi đè bằng 0 . Hệ quả : dữ liệu như mật khẩu, mã khóa lưu trên ngăn xếp vẫn có thể bị kẻ tấn công đọc sau khi thủ tục kết thúc nếu vùng nhớ không được chủ động ghi đè bằng 0

C. Thuật toán kiểm tra ngoặc bằng ngăn xếp

Ý tưởng: duyệt chuỗi từ trái sang phải; gặp ngoặc mở thì push; gặp ngoặc đóng thì pop ngoặc mở ở đỉnh để đối chiếu loại; cuối chuỗi ngăn xếp phải rỗng. Bảng 12 minh họa cách ngăn xếp thay đổi với một chuỗi hợp lệ (đáy ở bên trái, đỉnh ở bên phải).

Bảng 12. *Vết thực thi với chuỗi hợp lệ* `[(x)]`

Vị trí	Ký tự	Hành động	Ngăn xếp sau hành động
1	`[`	push `[`	`[`
2	`(`	push `(`	`[ (`
3	`x`	bỏ qua	`[ (`
4	`)`	pop được `(`, cùng loại với `)`	`[`
5	`]`	pop được `[`, cùng loại với `]`	(rỗng)
Hết chuỗi		ngăn xếp rỗng	hợp lệ

Bước 3.2 (DỰ ĐOÁN). Lập vết thực thi cho chuỗi `{a[b)}` bằng cách điền Bảng 13 theo mẫu Bảng 12. Dừng ở ký tự đầu tiên làm chuỗi sai; ghi kết quả cuối cùng là vị trí lỗi và kiểu lỗi (mô tả bằng lời; mã lỗi được quy định ở phần D).

Bảng 13. *Vết thực thi với chuỗi* `{a[b)}`

Vị trí	Ký tự	Hành động	Ngăn xếp sau hành động
1	`{`	push	{
2	`a`	Bỏ qua	{
3	`[`	Push	{[
4	`b`	Bỏ qua	{[
5	`)`	pop	{
Kết luận		Lỗi ở vị trí 5: ngoặc đóng không cùng loại với ngoặc mở ở đỉnh (mã 1), kết quả (5, 1)	

D. Nhiệm vụ: thủ tục check_brackets

Bước 3.3. Cài đặt thủ tục `check_brackets` theo đặc tả sau (không có mã khung; tự thiết kế thuật toán và cách nhận biết ngăn xếp rỗng).

Bảng 14. *Đặc tả* `check_brackets`

Mục	Yêu cầu
Tham số	`$a0` = địa chỉ chuỗi `.asciiz`.
Kết quả	`$v0` = vị trí (đếm từ 1, tính mọi ký tự) và `$v1` = mã lỗi, theo Bảng 15.
Ngăn xếp dữ liệu	Dùng chính ngăn xếp của chương trình (`$sp`); mỗi phần tử chiếm một word (4 byte). Là thủ tục lá.
Ràng buộc	Khi return, `$sp` phải bằng giá trị lúc vào thủ tục **trên mọi đường thoát**, kể cả khi phát hiện lỗi giữa chừng lúc ngăn xếp còn phần tử.

Bảng 15. *Mã lỗi và giá trị trả về*

`$v1`	Ý nghĩa	`$v0`	Ví dụ
0	Chuỗi hợp lệ	0	`[(x)]` → (0, 0)
1	Ngoặc đóng không cùng loại với ngoặc mở đang ở đỉnh ngăn xếp	vị trí của ngoặc đóng đó	`(x]` → (3, 1)
2	Ngoặc đóng khi ngăn xếp rỗng	vị trí của ngoặc đóng đó	`x)` → (2, 2)
3	Hết chuỗi nhưng ngăn xếp còn ngoặc mở	độ dài chuỗi + 1	`[x` → (3, 3)

Bước 3.4 (DỰ ĐOÁN → QUAN SÁT). Với mỗi chuỗi trong Bảng 16, tính tay cặp (`$v0`, `$v1`) trước, sau đó chạy. Chuỗi P là chuỗi cá nhân: gồm **k** cặp ngoặc lồng nhau hoàn toàn (k lấy từ Bảng 4), các ngoặc mở lần lượt theo chu kỳ `(`, `[`, `{`, `(`, `[`, `{`, … và các ngoặc đóng đúng loại, đúng thứ tự.

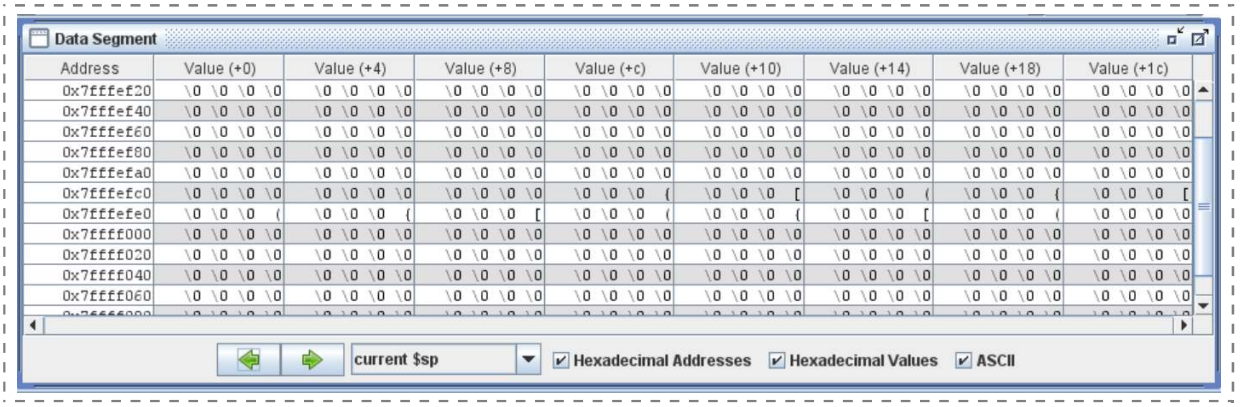
Bảng 16. *Kết quả kiểm tra ngoặc*

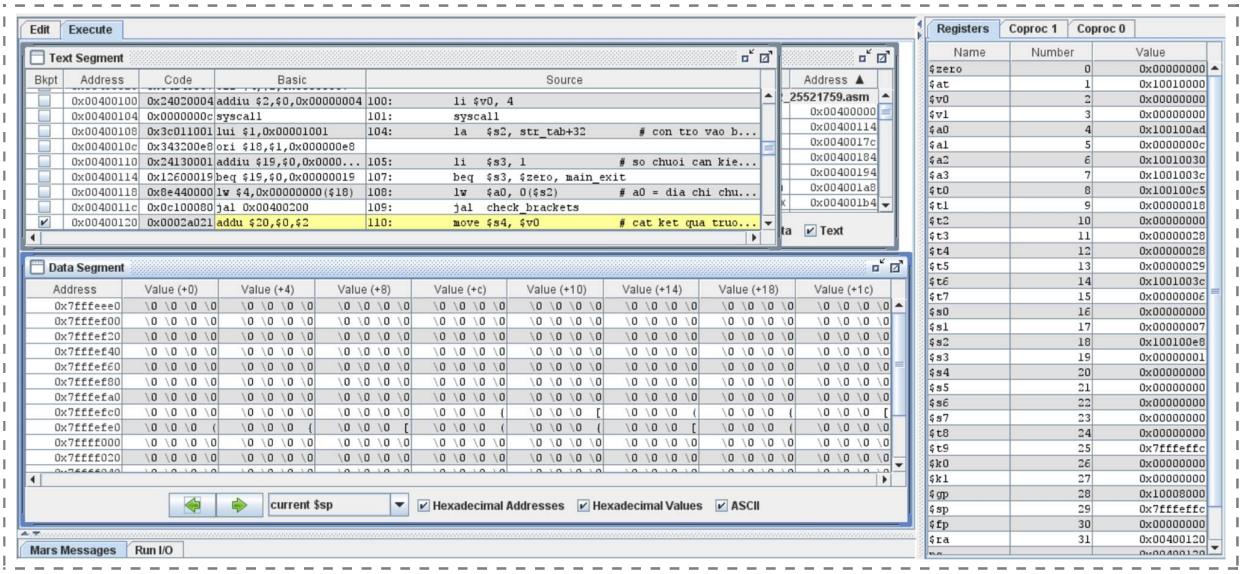
Bộ	Chuỗi	(`$v0`, `$v1`) tính tay	Chương trình
1	`(a+b)*[c-{d/e}]`	(0,0)	(0,0)
2	`f(x[i]{j}))`	(11,2)	(11,2)
3	`{[(])}`	(4,1)	(4,1)
4	`((a)`	(5,3)	(5,3)
5	"" (rỗng)	(0,0)	(0,0)
6	`no brackets here`	(0,0)	(0,0)
7	`]`	(1,2)	(1,2)
8	`([)]`	(3,1)	(3,1)
P	chuỗi cá nhân	(0,0)	(0,0)

Bước 3.5 (DỰ ĐOÁN → QUAN SÁT). Với chuỗi P, đặt breakpoint tại lệnh đầu tiên xử lý một ngoặc đóng (tức là lúc ngăn xếp sâu nhất). Điền Bảng 17 và dán ảnh Data Segment ở chế độ "current $sp" tại thời điểm đó; sau đó chạy tiếp đến khi thủ tục trả về và chụp lại cùng vùng nhớ.

Bảng 17. *Độ sâu ngăn xếp với chuỗi P*

Đại lượng	Dự đoán	Quan sát
Số phần tử trên ngăn xếp lúc sâu nhất	12	12
`$sp` lúc sâu nhất (hex)	0x7fffefcc	0x7fffefcc
`$sp` sau khi thủ tục trả về (hex)	0x7fffeffc	0x7fffeffc





Câu 3.2 (Đánh giá). Nêu ít nhất hai cách khôi phục `$sp` trên các đường thoát có lỗi (khi ngăn xếp còn phần tử). So sánh về số lệnh, độ phức tạp và rủi ro sai sót, rồi nêu cách bạn đã chọn và lý do.

Cách 1 (em chọn): lưu $sp lúc vào. move $t9,$sp ở đầu và move $sp,$t9 tại lối ra chung. Chỉ thêm 2 lệnh, chi phí O(1) không phụ thuộc độ sâu. Mọi đường thoát đều đi qua một chỗ nên khó quên, rủi ro thấp. Thanh ghi $t9 còn dùng luôn để kiểm tra ngăn xếp rỗng.

Cách 2: pop từng phần tử trong vòng lặp đến khi rỗng. Chi phí O(độ sâu), tức khoảng 3–4 lệnh mỗi phần tử còn lại, chậm nhất và cũng dài nhất.

Em chọn cách 1 vì ít lệnh nhất, thời gian không đổi, và gom mọi lối ra về một điểm duy nhất nên đảm bảo $sp cân bằng trên mọi đường thoát.

Câu 3.3 (Phân tích). Nếu mỗi phần tử chỉ chiếm một byte (dùng `sb` và giảm `$sp` đi 1), chuỗi P tiết kiệm được bao nhiêu byte ngăn xếp? Phương án đó vi phạm đặc điểm nào của `$sp` đã nêu ở phần A, và gây lỗi gì nếu sau đó dùng `lw` hoặc `sw` với địa chỉ dựa trên `$sp`?

Với P, lúc sâu nhất ngăn xếp chứa 12 phần tử: dùng word cần 12×4 = 48 byte, dùng byte chỉ cần 12 byte, nên tiết kiệm 36 byte. Nhưng phương án này vi phạm đặc điểm "$sp luôn là bội số của 4". Sau một lần push byte, $sp = 0x7fffeffb, không còn căn lề word. Nếu sau đó dùng lw/sw với địa chỉ dựa trên $sp (ví dụ một thủ tục khác push $ra), MARS sẽ báo lỗi *"address not aligned on word boundary"*, giống hệt hiện tượng ở Bước 1.4 của TN1.

PHẦN MỞ RỘNG – TRỰC QUAN HÓA PHỔ ĐIỂM (15 phút · cộng 1,0 điểm)

Chỉ thực hiện khi đã hoàn thành các thí nghiệm trên. Viết thủ tục `draw_hist` vẽ phổ điểm dưới dạng biểu đồ cột ngang bằng ký tự `|`: mỗi nhóm xếp loại là một dòng gồm nhãn nhóm, rồi số ký tự `|` bằng số sinh viên của nhóm đó. Ví dụ định dạng (dữ liệu minh họa, không phải kết quả của bạn):

```
g0: |||
g1: |||||
g2: ||||||||
g3: ||||
```

Thủ tục	Tham số	Yêu cầu
`draw_hist`	`$a0` = địa chỉ mảng `hist` `$a1` = số nhóm (4)	In đúng định dạng trên cho dữ liệu do `histogram` tính được; nhóm không có sinh viên vẫn in dòng nhãn, không có ký tự `|`. Là thủ tục lá; chương trình chính gọi sau `histogram`.

Lưu ý: các syscall in ký tự và in số dùng `$v0` và `$a0` để nhận mã và tham số.

Dán ảnh cửa sổ Run I/O với mảng điểm cá nhân (bộ T1):



KẾT THÚC – NỘP BÀI, PHẢN TƯ VÀ VẤN ĐÁP (20 phút · 2,0 điểm)

Danh sách kiểm tra trước khi nộp

☐	Mục
☐	File `CE119_Lab02_<MSSV>.asm` chạy được từ đầu đến cuối, kết thúc bằng syscall thoát.
☐	Mỗi thủ tục có chú thích đầu (tham số, giá trị trả về, thanh ghi sử dụng) và tuân thủ quy ước thanh ghi.
☐	Báo cáo `CE119_Lab02_<MSSV>.docx` đã điền đủ các bảng, câu hỏi và ảnh chụp.
☐	Phần "Liên kết GitHub" và "Tự chấm" ở đầu trang đã điền.

Câu hỏi phản tư

Câu P1 (Đánh giá). Nêu hai lỗi đáng kể nhất bạn gặp trong buổi thực hành. Với mỗi lỗi: nguyên nhân gốc (gắn với quy ước thanh ghi, bộ nhớ hoặc ngăn xếp), công cụ MARS giúp bạn phát hiện, và cách bạn sửa.

Lỗi 1, in sai min/max: sau li $v0,4 để in chuỗi, em vẫn dùng move $a0,$v0 để in min, nên in ra 4 thay vì 0. Nguyên nhân: $v0 vừa là giá trị trả về vừa là mã syscall nên bị ghi đè (quy ước thanh ghi). Phát hiện nhờ Run I/O cho kết quả sai, rồi Step và theo dõi Registers. Sửa: cất kết quả vào $s0, $s1 ngay sau jal stats và in từ đó.

Lỗi 2, histogram dồn hết vào g0: lbu ghi vào $t3 nhưng sll lại dùng $t4 (chưa gán, bằng 0), nên luôn tăng hist[0]. Phát hiện nhờ breakpoint tại sll: Registers cho thấy $t4 = 0, Data Segment cho thấy chỉ hist[0] tăng. Sửa: đổi đích của lbu thành $t4.

Câu P2 (Đánh giá). Để `check_brackets` báo được vị trí của ngoặc mở chưa đóng (mã 3) thay vì "độ dài chuỗi + 1", cần thay đổi gì ở phần tử đẩy lên ngăn xếp? Ước lượng ảnh hưởng đến bộ nhớ ngăn xếp của chuỗi P.

Cần lưu thêm vị trí của ngoặc mở khi push. Cách 1: push 2 word (ký tự và vị trí). Với P, lúc sâu nhất cần 12 × 8 = 96 byte, gấp đôi 48 byte hiện tại. Cách 2: gộp vào 1 word, vị trí ở 24 bit cao, ký tự ở 8 bit thấp (sll + or khi push, andi + srl khi pop). Bộ nhớ vẫn 48 byte, chỉ thêm khoảng 2 lệnh mỗi lần push/pop. Khi gặp mã 3, đọc vị trí ở phần tử đỉnh 0($sp). Em chọn cách 2.

Câu P3 (Đánh giá). Thủ tục `stats` trả về hai giá trị qua `$v0` và `$v1`. Giả sử cần thêm kết quả thứ ba (ví dụ tổng điểm) trong khi quy ước chỉ có hai thanh ghi trả về. Đề xuất ít nhất hai cách và nêu ưu, nhược điểm của từng cách về số lệnh và bộ nhớ.

Cách 1, tham số ra qua bộ nhớ: truyền địa chỉ biến sum qua $a2, thủ tục sw kết quả vào đó. Chỉ thêm khoảng 3 lệnh và 4 byte, nhưng phải truy cập bộ nhớ.

Cách 2, trả về qua ngăn xếp: người gọi dành 4 byte trên ngăn xếp, thủ tục ghi vào. Không cần biến toàn cục, nhưng tốn khoảng 4 lệnh và phải giữ $sp cân bằng.

Cách 3, tách thủ tục sum_array riêng: code gọn, nhưng phải duyệt mảng thêm một lượt (khoảng 6n lệnh).

Em chọn cách 1 vì ít lệnh nhất và vẫn duyệt mảng một lượt.

Vấn đáp. Giảng viên có thể hỏi bất kỳ phần nào của mã và báo cáo đã nộp; sinh viên cần giải thích được lựa chọn thiết kế và tái hiện được kết quả quan sát trên MARS.